Lettre aux lycéennes et aux lycéens

Comprendre NGeGenBrain_v1 : ce qu'il fait, ce qu'il ne fait pas et ce qu'il pourrait devenir

Document pédagogique fondé sur l'examen de l'archive logicielle `Dmodel_NGeGenBrain_v1.zip`
Version étudiée : 1.0.0 ; archive datée du 27 octobre 2026

Chères lycéennes, chers lycéens,

Vous avez peut-être déjà entendu parler d'intelligence artificielle, de robots ou de systèmes capables de répondre à des questions. Le nom `NGeGenBrain_v1` peut donner l'impression qu'il s'agit d'un cerveau artificiel. Pour comprendre le projet correctement, commençons par une règle simple : **on ne doit pas déduire les capacités d'un logiciel de son nom ; il faut regarder son code, le faire fonctionner et tester ses résultats.**

Cette lettre vous présente donc le système qui se trouve réellement dans l'archive examinée. Elle distingue trois choses : ce que la version actuelle sait faire dans son périmètre, ce que les tests permettent d'affirmer, et ce qu'une version future pourrait éventuellement accomplir si elle était développée et évaluée pour cela.

1. Qu'est-ce que NGeGenBrain ?

`NGeGenBrain_v1` est un logiciel exécutable localement, écrit en Python, qui organise des opérations informatiques à partir de composants enregistrés, de règles explicites et de données structurées. Il associe notamment :

— un moteur qui exécute des programmes décrits sous forme de données JSON ;

— un *maillage* de faits et de relations, dans lequel des règles déclarées peuvent produire de nouvelles conclusions ;

— des contrôles de types, de permissions et de budgets de calcul ;

— une mémoire persistante et des traces qui permettent de retrouver ce qui s'est passé ;

— des outils de recherche de candidats, de synthèse de petits programmes et d'évaluation de modèles simples du monde ;

— une interface en ligne de commande et une interface Web destinée à fonctionner sur la machine locale.

Le mot *compositionnelle* signifie ici que le système peut construire un traitement en assemblant des opérations plus petites, un peu comme on construit une phrase à partir de mots ou une recette à partir d'étapes. Le type d'une donnée indique sa catégorie attendue : par exemple un nombre entier, un texte ou un objet structuré. Un contrat précise ce qu'une opération accepte et ce qu'elle doit renvoyer.

Il est donc plus exact de décrire la version actuelle comme **un moteur de calcul et de raisonnement symbolique contrôlé, avec mémoire et traces**, plutôt que comme un cerveau comparable au nôtre. Le projet possède des mécanismes qui peuvent évoquer certaines fonctions cognitives, mais cette ressemblance ne démontre ni conscience, ni compréhension humaine, ni intelligence générale.

2. Quel est son but ?

Son but pratique est de rendre des traitements composés plus organisés, vérifiables et contrôlables. Au lieu de demander au programme de faire confiance à une réponse simplement parce qu'elle paraît plausible, le système tente de conserver les éléments qui permettent de comprendre comment un résultat a été produit.

On peut résumer son principe ainsi :

Décrire le travail → vérifier les règles → exécuter dans des limites → enregistrer le résultat et sa trace

Ce choix est utile lorsqu'un résultat doit être inspecté : quelles données ont été utilisées ? Quelle règle a permis une déduction ? Le calcul a-t-il été terminé ? Le système a-t-il refusé une action, rencontré une limite ou seulement produit une simulation ? Les réponses à ces questions comptent autant que la valeur finale.

3. Comment fonctionne-t-il, étape par étape ?

1. **On décrit une tâche.** Le programme est représenté par un arbre de syntaxe abstraite (AST) sérialisé en JSON. Il s'agit d'une description structurée des opérations à effectuer, pas d'un morceau de code Python libre à exécuter.
2. **Le système vérifie la demande.** Il contrôle la forme des données, les types attendus, les capacités autorisées, les préconditions et les budgets déclarés. Une tâche qui ne respecte pas les conditions peut être rejetée.
3. **Il appelle des composants enregistrés.** Ces composants réalisent les opérations prévues. Le moteur vérifie ensuite les résultats et les postconditions annoncées.
4. **Il conserve les éléments utiles.** La base SQLite peut conserver des faits, des règles, des éléments de mémoire, des métadonnées et des traces. La trace d'événements est chaînée par empreintes pour aider à détecter certaines modifications.
5. **Il rend un statut explicite.** Un résultat peut être réussi, partiel, non résolu, rejeté, échoué ou annulé. Il ne faut donc pas confondre ń une valeur existe ż et ń la tâche a été menée à bien ż.

Une métaphore : un atelier avec un cahier de laboratoire

Imaginez un atelier scolaire. Les outils sur l'établi sont les composants ; les consignes de sécurité sont les contrats, les permissions et les budgets ; le cahier de laboratoire est la mémoire et la trace. Lorsqu'une expérience échoue, le cahier ne devrait pas écrire ń réussite ż simplement pour faire plaisir : il doit indiquer l'échec, le résultat partiel ou la limite rencontrée.

Cette métaphore aide à comprendre l'organisation du logiciel, mais elle a une limite importante : un logiciel ne devient pas scientifique uniquement parce qu'il garde un journal. Il faut aussi que ses données soient fiables, que ses règles soient adaptées et que ses conclusions soient validées.

4. Que peut faire la version actuelle ?

Les capacités ci-dessous sont présentes dans le code et décrites dans le guide du projet. Cela ne signifie pas que chaque usage possible a été validé dans des conditions réelles.

Fonction	Ce qu'elle permet actuellement
Exécution compositionnelle	Assembler et exécuter des opérations déclarées dans un AST JSON, avec contrôles de contrats, de capacités et de budgets.
Maillage de faits	Enregistrer des faits structurés, interroger des relations et appliquer des règles déclarées comportant une ou plusieurs prémisses.
Dérivations et certificats	Associer certaines conclusions à leurs prémisses et à leurs règles, puis revérifier cette dérivation contre les éléments actifs.
Mémoire persistante	Stocker et retrouver des données locales, créer des points de contrôle, suspendre/réactiver certains éléments et tenir compte des dépendances ou révocations connues.
Traces et intégrité	Journaliser des événements, chaîner leurs empreintes et vérifier l'intégrité de la base et de la chaîne d'événements.
Synthèse bornée	Chercher des programmes candidats cohérents avec des exemples d'entrée et de sortie, sous un budget de recherche.
Modèles du monde simples	Définir une fonction de transition, produire une prédiction simulée et évaluer des prédictions sur un jeu de contrôle distinct, comparé à une référence.
Intentions conversationnelles	Associer des motifs de texte pré-définis à des paramètres typés et à des programmes déclaratifs. Une phrase ambiguë peut rester non résolue.
Interfaces	Utiliser une ligne de commande, un service HTTP local et une interface Web qui partagent le même catalogue d'actions métier.

Exemple : relier deux informations

Supposons que le maillage contienne les deux faits observés suivants :

— ń A est relié à B ż ;

— ń B est relié à C ż.

Si une règle de transitivité a été déclarée explicitement, le système peut alors dériver ń A est relié à C ż. Il peut enregistrer les faits utilisés et la règle ayant produit la conclusion. La règle n'est pas automatique par magie : elle doit être déclarée, et la conclusion n'est valable que dans le cadre des prémisses et de la règle employées.

De même, la relation ń A est relié à B ż ne permet pas à elle seule de conclure ń B est relié à A ż. Il faut une autre règle qui autorise cette inversion. Cette distinction est essentielle : **un système rigoureux ne doit pas inventer une règle simplement parce qu'elle semble raisonnable.**

5. Ce que les tests permettent de dire

La suite de tests du dépôt V1 a été exécutée à partir d'une extraction de l'archive fournie. Dans cet environnement, elle a produit **65 tests réussis, 0 échec et 0 erreur**. Le rapport d'audit présent dans l'archive présente en outre un benchmark indépendant de 15 scénarios ; il rapporte 15 réussites sur 15 pour la version V1, après 8 réussites et 7 échecs sur les 15 mêmes scénarios pour la version V0.

Ces tests couvrent notamment l'arithmétique rationnelle, la fermeture de certains graphes, la gestion des types et des contextes, des limites de ressources, la validation des règles, la résistance à la falsification d'un certificat, la manipulation d'entrées profondément imbriquées et le refus d'exécuter du code Python présenté comme simple texte.

Il s'agit d'éléments encourageants : des défauts observés dans la version antérieure ont été corrigés et des tests de non-régression ont été ajoutés. Mais un test vérifie un cas déterminé ou une propriété dans une portée déterminée. **65 tests réussis ne prouvent pas que toutes les entrées possibles, toutes les attaques ou tous les usages futurs seront sûrs.** Un

benchmark fini n'est ni une preuve mathématique universelle, ni une certification de sécurité industrielle.

6. Ce que NGeGenBrain ne sait pas démontrer aujourd'hui

Il est aussi important de connaître les limites que les fonctions présentes dans le code :

— **Ce n'est pas un assistant conversationnel général.** Le module de chat résout des motifs déclarés et des emplacements typés. Il ne comprend pas automatiquement toutes les phrases, les sous-entendus, l'humour ou les questions inconnues.

— **Une prédiction reste une simulation.** Les résultats des modèles du monde sont étiquetés comme simulés, non observés et non certifiés. Un bon résultat sur un échantillon de contrôle ne garantit pas que le modèle fonctionnera dans le monde réel.

— **Une synthèse cohérente avec des exemples n'est pas une preuve universelle.** Un programme qui réussit sur quelques exemples peut échouer sur une entrée jamais testée.

— **Un certificat ne prouve pas que les prémisses sont vraies.** Il peut aider à contrôler qu'une conclusion suit de prémisses et de règles enregistrées ; il ne transforme pas des données fausses en vérités.

— **Une empreinte ne garantit pas la véracité.** Une chaîne d'événements ou un hash peut aider à détecter une altération, mais ne prouve pas que l'événement initial était exact ni qui l'a écrit.

— **Le projet n'est pas livré comme un système distribué ou un service public durci.** L'interface HTTP est conçue pour la machine locale et refuse par défaut les adresses d'écoute non locales. Elle ne doit pas être exposée directement à Internet comme un service multi-utilisateur de production.

— **Toutes les affirmations théoriques ne sont pas prouvées par machine.** Le dépôt contient des textes formels et des obligations de raffinement, mais cela ne revient pas à une vérification complète de chaque exécution du programme Python par un assistant de preuve.

Une autre prudence concerne ASCR : le code contient une reconstruction nommée ASCR-NG pour un rôle de recherche/récupération, mais les sources historiques du runtime ASCR/AAM n'étaient pas présentes dans les éléments examinés. L'équivalence exacte avec une implémentation historique inconnue n'est donc pas démontrée.

7. À quoi pourrait servir la version actuelle ?

Dans son périmètre réel, NGeGenBrain peut servir de base expérimentale ou pédagogique, notamment pour :

— apprendre comment on construit un programme à partir d'opérations plus simples ;

— illustrer les différences entre un fait enregistré, une hypothèse, une déduction et une simulation ;

— explorer des graphes de relations et des systèmes de règles explicites ;

— étudier les types, les contrats, les budgets de calcul, la traçabilité et la gestion d'erreurs ;

— réaliser des prototypes locaux pour des tâches structurées dont les règles sont connues et que l'on peut tester ;

— comparer une prédiction simple à une référence sur des données tenues à l'écart de la construction du modèle.

Exemples scolaires possibles : suivre des relations dans un réseau de transport imaginaire, appliquer une règle logique à une petite base de faits, simuler une transition numérique ou examiner pourquoi une action a été rejetée. Ces exemples sont des usages proposés, non des modules scolaires spécialisés livrés automatiquement.

Pour le moment, il serait imprudent de lui confier seul des décisions médicales, juridiques, financières, de sécurité, de recrutement ou toute tâche dont l'échec pourrait blesser quelqu'un ou lui causer un tort important. Ces domaines demandent des données fiables, des évaluations spécifiques, des contrôles humains et des garanties bien plus fortes que les tests logiciels présentés ici.

8. Que pourrait accomplir une version plus avancée ?

Le code fournit une architecture sur laquelle de nouvelles fonctions pourraient être construites, mais une possibilité théorique n'est pas une capacité déjà disponible. Avec du développement, des données adaptées, des outils supplémentaires et une évaluation indépendante, des versions futures pourraient par exemple :

— **mieux traduire le langage naturel en tâches structurées**, avec un mécanisme capable de demander des précisions quand une demande est ambiguë ;

— **se connecter à des sources de données ou à des capteurs**, tout en conservant l'origine, la date et le niveau de confiance des informations ;

— **construire des simulations plus riches** pour comparer des scénarios et analyser les écarts entre prédiction et observation ;

— **planifier une suite d'actions** sous contraintes, puis vérifier chaque étape avant de l'exécuter ;

— **aider à créer et tester des programmes**, en recherchant des candidats, générant des cas de test et présentant les contre-exemples trouvés ;

— **intégrer un vérificateur formel externe** pour certaines propriétés mathématiques ou logicielles, si les propriétés sont correctement spécifiées et si la chaîne de vérification est elle-même validée ;

— **prendre en charge des collaborations ou déploiements distribués**, à condition de développer l'authentification, l'isolation, la gestion des accès et les mécanismes de sécurité nécessaires.

Ces améliorations ne sont pas garanties par la seule architecture. Par exemple, ajouter un modèle de langage pourrait rendre le dialogue plus souple, mais ne garantirait pas que toutes ses réponses soient exactes. Ajouter des capteurs pourrait fournir davantage de données, mais ne garantirait pas qu'elles soient justes. Connecter un robot ne donnerait pas automatiquement au système une compréhension sûre du monde physique.

Pour qu'une nouvelle capacité devienne crédible, il faudrait la définir précisément, l'implémenter, la tester sur des situations variées, publier ses limites, examiner les échecs et prévoir un moyen d'arrêter ou de reprendre la main.

9. Conséquences : les avantages possibles et les risques

Les avantages possibles

— **Plus de visibilité :** une trace peut faciliter la compréhension d'un résultat et le diagnostic d'une erreur.

— **Des refus explicites :** les contrats, capacités et budgets peuvent empêcher certaines opérations non autorisées ou trop coûteuses dans le périmètre contrôlé.

— **Des raisonnements plus inspectables :** des faits, des règles et leurs dépendances permettent d'examiner comment une conclusion a été obtenue.

— **Une expérimentation locale :** le système n'a pas besoin d'un service cloud externe pour son fonctionnement décrit ; ses données persistantes restent toutefois sur la machine où il est installé.

Les risques à ne pas ignorer

— **Le risque de confiance excessive :** une interface soignée ou un certificat technique peut donner une impression de certitude supérieure aux preuves réelles.

— **Les données erronées :** une déduction impeccable à partir de prémisses incorrectes peut donner une conclusion incorrecte.

— **La confidentialité :** la base locale peut contenir des messages, des faits, des traces ou des métadonnées. ń Local ż ne signifie pas automatiquement ń protégé ż ; il faut sécuriser l'appareil, les comptes, les sauvegardes et l'accès au fichier.

— **Les biais et les erreurs de conception :** les règles choisies, les données sélectionnées et les critères d'évaluation influencent les résultats.

— **L'automatisation de mauvaises décisions :** une erreur répétée à grande vitesse peut amplifier ses conséquences si personne ne contrôle le processus.

— **La maintenance et la responsabilité :** il faut garder les versions, les journaux et les tests, et déterminer qui peut modifier les règles, autoriser une action ou corriger un résultat.

La bonne attitude n'est donc ni d'avoir peur de toute intelligence artificielle, ni de croire qu'un système informatique est fiable parce qu'il porte un nom impressionnant. C'est de comprendre ce qu'il fait, d'exiger des preuves adaptées à l'usage et de garder une personne responsable lorsque les conséquences sont importantes.

10. Comment évaluer une prochaine version ?

Avant de dire qu'une capacité est acquise, il est utile de poser au minimum les questions suivantes :

1. Quel comportement exact prétend-on obtenir et dans quelles conditions ?
2. Quelles entrées ont été testées, et lesquelles ne l'ont pas été ?
3. Existe-t-il une référence indépendante permettant de juger les résultats ?
4. Les erreurs, les sorties partielles et les refus sont-ils conservés et expliqués ?
5. Peut-on vérifier l'origine des données et reproduire l'expérience ?
6. Que se passe-t-il si les données sont fausses, contradictoires ou malveillantes ?
7. Qui autorise les actions, protège les données et assume les conséquences ?

Cette méthode est valable pour NGeGenBrain comme pour n'importe quel logiciel d'intelligence artificielle. Elle apprend à distinguer une démonstration convaincante d'une capacité réellement mesurée.

Pour conclure

Chères lycéennes, chers lycéens, NGeGenBrain_v1 peut être vu comme un atelier informatique qui sait assembler des opérations, manipuler des faits et des règles, conserver des traces et produire certaines déductions sous contrôle. Sa valeur actuelle ne vient pas d'une intelligence mystérieuse, mais d'une architecture explicite dont certaines propriétés ont été testées.

Son ambition future pourrait être plus grande : mieux comprendre des demandes, simuler des situations, aider à construire des programmes ou coordonner des outils. Mais chaque ambition exige du travail réel, des validations et une description honnête des limites. Il ne faut pas confondre ce que le logiciel fait aujourd'hui, ce que l'on espère lui faire faire demain et ce qu'il serait mathématiquement possible de lui faire faire dans des conditions idéales.

Retenez enfin cette image : **un résultat est comme une réponse écrite au tableau ; la méthode, les hypothèses et les vérifications sont le travail qui permet de savoir si cette réponse mérite notre confiance.** En informatique comme en sciences, la curiosité est essentielle, mais la rigueur consiste aussi à dire ń je ne sais pas encore ż quand les preuves manquent.

Avec confiance dans votre curiosité et votre esprit critique,

Lettre pédagogique destinée aux lycéennes et aux lycéens.

Repères techniques et portée de l'analyse

Cette annexe sert à rendre le document vérifiable. Elle ne remplace pas la documentation et les tests du projet.

Éléments directement examinés

— Archive analysée : `Dmodel_NGeGenBrain_v1.zip`.

— Empreinte SHA-256 calculée sur l'archive examinée :

`71534f98ff590e5c1a103888f0e553e9e8966d34d7f0364429eac8eae238b5c8`.

— Guide utilisateur : `Dmodel_NGeGenBrain_v1/docs/USER_GUIDE.md`.

— Rapport d'audit : `Dmodel_NGeGenBrain_v1/docs/REPORT_AUDIT_NGeGenBrain_v1.md`.

— Audit du formalisme : `Dmodel_NGeGenBrain_v1/docs/FORMALISM_AUDIT.md`.

— Code principal : répertoire `Dmodel_NGeGenBrain_v1/src/ngegenbrain/` ; tests : `Dmodel_NGeGenBrain_v1/tests/`.

Résultats observés et interprétation

La suite fournie a été exécutée depuis une extraction de l'archive avec la commande :

```
PYTHONWARNINGS=error::ResourceWarning PYTHONPATH=src \
python3 -m unittest discover -s tests -v
```

Elle a terminé avec le résultat `Ran 65 tests` puis `OK`. Le rapport d'audit du projet signale séparément un benchmark indépendant de 15 scénarios, tous réussis sur V1. Ce dernier résultat est rapporté comme résultat du banc inclus dans la chaîne d'audit ; l'exécution présente de cette analyse a rejoué la suite de 65 tests, pas l'ensemble complet du banc indépendant externe.

Les conclusions de cette lettre sont donc limitées au code, à la documentation et aux tests énoncés. Elles ne prétendent pas prouver l'absence de tout défaut, la sécurité absolue, la correction de tout programme possible, la conscience du système ou une équivalence historique avec des composants dont les sources n'étaient pas présents.

Glossaire rapide

AST	Représentation structurée d'un programme sous forme d'arbre, ici encodée en JSON.
Contrat	Conditions décrivant les entrées autorisées, les sorties attendues et certaines obligations d'une opération.
Maillage	Réseau de faits et de relations, sur lequel des règles explicites peuvent produire des faits dérivés.
Prémisse	Information de départ nécessaire à une règle ou à une déduction.
Certificat	Structure permettant de vérifier une dérivation donnée contre des prémisses et des règles ; ce n'est pas une garantie que les prémisses sont vraies.
Holdout	Partie des données tenue à l'écart de la construction ou du réglage du modèle afin d'évaluer ses résultats sur des exemples distincts.
Simulation	Résultat calculé par un modèle ; il doit rester distinct d'une observation réelle.
Traçabilité	Capacité à retrouver les opérations, les données ou les règles qui ont participé à un résultat.